AI ENGINEERING / EVALUATION / TESTING / CASE STUDY

CASE STUDY

12

WORKFLOW EVALUATION HARNESS

A delivery team swaps hand picked spot checks for a repeatable release gate on AI changes.

PROPOSED RESEARCH / NOT YET VERIFIED

THE BUSINESS CONTEXT

A delivery team maintains several AI assisted workflows for clients. Prompts and models and rules change often. Each change is judged by a handful of examples and a quick read of the output. Nobody can say whether a change made the workflow better or worse overall.

WHO IS INVOLVED AND WHAT THEY NEED

01	**Delivery lead**	Needs confidence that a change is safe before it reaches a client process.
02	**AI engineer**	Needs fast repeatable feedback on whether a change helped or caused regressions.
03	**Client operations owner**	Needs assurance that accuracy and speed and cost stay within agreed bounds.

SCOPE AND HONEST LIMITS

- ⚠ This is a proposed design on synthetic data and no results or savings are claimed.
- ⚠ A passing gate lowers risk but does not prove a workflow is safe in live use.
- ⚠ Synthetic cases cannot cover every real input so the library needs steady growth.
- ⚠ Scores from model outputs vary and need repeat trials and stated uncertainty.

CURRENT WORKFLOW AND FAILURE POINTS

12

PROPOSED RESEARCH / NOT YET VERIFIED

HOW THE WORK RUNS TODAY

1 TODAY

Edit the workflow

An engineer changes a prompt or model setting or rule in the pipeline.

2 TODAY

Try a few examples

Five or six remembered cases are pasted in and the output is read by eye.

3 TODAY

Judge by impression

If the examples look right the change is treated as good.

4 TODAY

Ask for a second look

A colleague skims the same examples and agrees in a message thread.

5 TODAY

Release to the client

The change goes live and the team watches for complaints.

6 TODAY

Investigate complaints

A user reports a bad result and the team works back to find which change caused it.

WHERE IT BREAKS

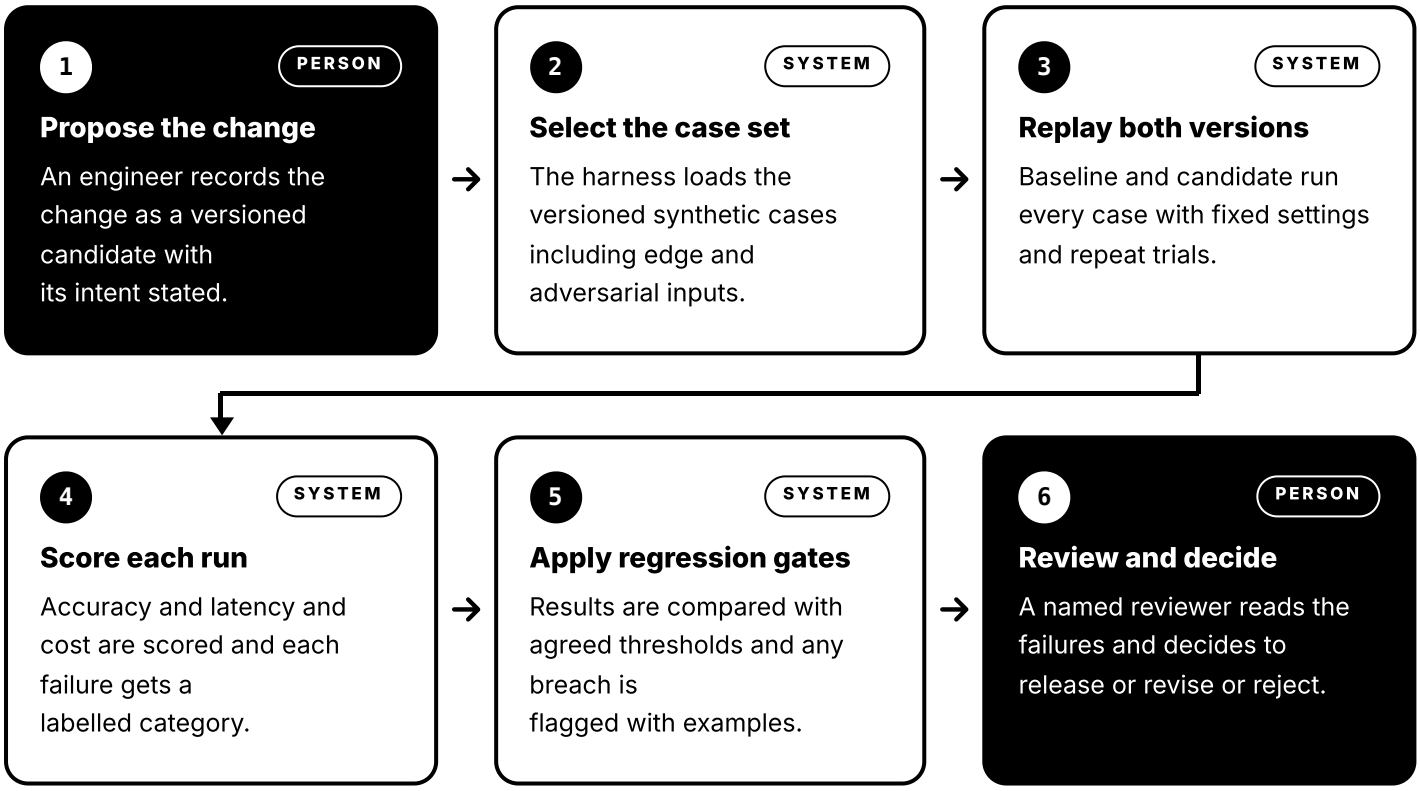
Unrepeatable checks	The examples differ each time so two changes cannot be compared fairly.
Hidden regressions	A fix for one case quietly breaks others that nobody reran.
Unseen cost and delay	Token spend and response time drift upward without anyone measuring them.
Late discovery	Failures are found by clients after release and not by the team before it.

REDESIGNED WORKFLOW AND DECISION AUTHORITY

12

PROPOSED RESEARCH / NOT YET VERIFIED

THE REDESIGNED WORKFLOW



SYSTEM STEP PERSON DECISION

WHERE AUTHORITY STAYS WITH A PERSON

Release a workflow change	DELIVERY LEAD	A passing gate informs the decision but a person owns the consequences for the client.
Set or change gate thresholds	DELIVERY LEAD	Thresholds encode how much risk the client accepts and need an accountable owner.
Add or retire a test case	AI ENGINEER	The case library defines what counts as correct and must be reviewed like code.

ARCHITECTURE, DATA AND CONTROLS

12

PROPOSED RESEARCH / NOT YET VERIFIED

SYSTEM ARCHITECTURE

1	**Case library**	Versioned synthetic cases with expected outcomes and labelled failure categories.
2	**Replay runner**	Runs baseline and candidate with pinned settings and seeded repeat trials.
3	**Scoring**	Field accuracy and abstention behaviour and latency percentiles and cost per case.
4	**Gate engine**	Configurable regression rules that compare candidate against baseline.
5	**Reporting**	Run history and failure examples and a release record kept for each change.

CORE DATA ENTITIES

- **Test case**
- **Candidate version**
- **Baseline version**
- **Run**
- **Score**
- **Gate rule**

RISKS AND THE CONTROL FOR EACH

Gate passes a harmful change →	Planted defect tests measure how many known faults the gates actually block.
Noisy scores mislead →	Repeat trials report spread so small differences are not read as real gains.
Test cases go stale →	Case library changes are versioned and reviewed and new failures become new cases.
Release without review →	Release requires a recorded reviewer decision whatever the gate result.

EVALUATION AND DELIVERY PLAN

12

PROPOSED RESEARCH / NOT YET VERIFIED

HOW SUCCESS IS MEASURED

Defect block rate	Share of deliberately planted defects that the gates stop before release.
False block rate	Share of sound changes that the gates wrongly stop.
Score stability	Spread of scores across repeat runs on identical cases and settings.
Failure coverage	Share of labelled failure categories represented in the case library.
Review effort	Time a reviewer needs to reach a release decision compared with the manual route.

TEST PLAN

1. **Inject known defects into candidate versions and confirm the gates block each one.**
2. **Rerun an identical candidate several times and record the spread of every score.**
3. **Replace the model with a weaker one and confirm accuracy and cost gates both react.**
4. **Remove a failure category from the library and confirm coverage reporting flags the gap.**

DELIVERY PLAN

01 ALIGN	02 UNDERSTAND	03 DESIGN	04 DELIVER	05 LAUNCH
Agree which workflows to cover and what accuracy and latency and cost bounds matter.	Review how changes are made and tested today and collect recurring failure types.	Specify the case format and scorers and gate rules and the release record.	Build the runner and scorers and a synthetic case library with planted defect tests.	Run the harness beside the manual route for a period and compare the decisions reached.
DECISION Which workflows go first	**DECISION** Failure categories to track	**DECISION** Initial gate thresholds	**DECISION** Case library sign off	**DECISION** When gates become mandatory

DELIVERABLES

- **Versioned synthetic case library**
- **Replay runner and scorer reference implementation**
- **Gate rule configuration and threshold rationale**
- **Planted defect test suite**
- **Release record and regression report template**